

Universidade do Algarve
Faculdade de Ciências e Tecnologia
Licenciatura em Engenharia Informática

Laboratório de Engenharia de Software 2025/2026

T11: Sistema de Gestão de compras



PL4 – Grupo 1

David Cruz - a80038
Tiago Vailant - a79922
Ricardo Soares - a80019
Tiago Lourenço -

Fevereiro – Maio 2026

Introdução

No âmbito da unidade curricular de Laboratório de Engenharia de Software, foi proposta a criação de um sistema de software completo que aplicasse boas práticas de engenharia de software, processos ágeis e decisões arquiteturais bem fundamentadas.

Este projeto foca-se na resolução do “Tema 11: Gestão de Compras” aplicado a uma clínica veterinária, resultando no desenvolvimento da plataforma “VetStock”. O principal objetivo deste sistema é centralizar, automatizar e simplificar a aquisição de bens materiais essenciais ao funcionamento da clínica — como rações, medicamentos e material terapêutico —, garantindo o controlo rigoroso de stocks e a avaliação contínua de fornecedores, sem incluir qualquer contratação de serviços externos.

A plataforma “VetStock” foi concebida sob uma arquitetura cliente-servidor moderna (React no frontend, Node.js/Express no backend e base de dados PostgreSQL com Prisma ORM) e encontra-se disponível online em www.vetstock.ONLINE

Este relatório descreve detalhadamente todo o processo de engenharia do sistema, cobrindo o levantamento de requisitos, a gestão ágil do projeto com Scrum e Jira, o controlo de versões, as opções tecnológicas, a estratégia de deployment e, por fim, as conclusões e perspetivas de trabalho futuro.

Visão geral e âmbito do sistema

Visão Geral

O “VetStock” é uma solução de software integrada desenhada especificamente para gerir o ciclo de vida de aprovisionamento e stocks de uma clínica veterinária. O sistema centraliza a comunicação operacional entre o armazém, o departamento de compras e a administração financeira. Através de um painel de controlo, Dashboard, os utilizadores conseguem monitorizar os níveis de stock em tempo real, acompanhar o progresso de encomendas enviadas aos fornecedores e identificar rapidamente necessidades de reposição, otimizando o fluxo logístico global da clínica.

Âmbito do Sistema

O âmbito funcional do “VetStock” foi formalmente estruturado e gerido através de Épicas no Jira. Cada épico delimita uma área de responsabilidade e funcionalidade do sistema:

- Gestão de Utilizadores (Épico SCRUM-76): Garante a segurança e controlo de acessos. Permite a autenticação e atribuição de perfis de utilizador com permissões diferentes no sistema, nomeadamente Administrador, Responsável de Stock e Responsável Financeiro.
- Gestão de Stock (Épico SCRUM-38): Compreende o catálogo completo de bens materiais da clínica. Monitoriza as quantidades em stock real e emite alertas quando estas descem abaixo do limite definido (stock mínimo).

- Gestão de Fornecedores (Épico SCRUM-39): Centraliza o registo de parceiros comerciais, incluindo dados de contacto, NIF, observações e estado de atividade (ativo/inativo).
- Relação Produto-Fornecedor (Épico SCRUM-40): Mapeia quais os produtos que cada fornecedor distribui e os respetivos preços negociados. Permite também definir, para cada produto, qual o fornecedor preferencial para reabastecimentos automatizados.
- Gestão de Pedidos de Compra (Épico SCRUM-41): Gere a identificação de necessidades. Suporta a criação de pedidos de compra em estado PENDENTE, que podem ser gerados manualmente ou de forma automatizada (quando o stock atinge o nível mínimo). Inclui o fluxo de aprovação ou rejeição de pedidos.
- Gestão de Encomendas (Épico SCRUM-42): Controla a emissão formal das ordens de compra enviadas aos fornecedores. Cada encomenda gera um código único e formatado (ex: ENQ-AAAA-XXX) e passa por estados como EMITIDA, ENVIADA, ENTREGUE ou CANCELADA.
- Gestão Financeira e Receção (Épico SCRUM-156): Gere a receção física de mercadorias em armazém (conferência de quantidades e controlo de qualidade), suportando entregas totais ou parciais. Integra também a Avaliação de Fornecedores com classificações de 1 a 5 estrelas em conformidade, pontualidade, comunicação, preço e qualidade.
- Dashboard e UI/Melhorias (Épicos SCRUM-116 e SCRUM-117): Garantem a visibilidade agregada de KPIs (produtos em rutura, total de despesas, taxas de cumprimento dos fornecedores) e fornecem uma interface de utilizador reativa, intuitiva e com transições fluidas.

Atores do sistema

O sistema possui três atores principais:

Administrador

Perfil: É o utilizador com privilégios totais de gestão técnica do sistema.

Principais Responsabilidades:

- Gestão de Perfis e Contas: Criação, edição, ativação e suspensão de contas de utilizadores (incluindo alteração de palavras-passe).
- Supervisão Global: Acesso completo a todas as áreas operacionais, fornecedores, catálogo e histórico transacional para fins de auditoria ou assistência.

Responsável de Stock

Perfil: É o utilizador que opera diretamente no armazém da clínica, lidando com a gestão física de produtos e consumíveis.

Principais Responsabilidades:

- **Gestão do Catálogo:** Registrar, atualizar ou desativar produtos físicos, definindo os seus limites de stock mínimo.
- **Identificação de Necessidades:** Monitorizar produtos em rutura e iniciar pedidos de compra (manuais ou automáticos) para reposição.
- **Receção de Mercadorias:** Efetuar o registo físico da receção de encomendas entregues pelos fornecedores, conferindo as quantidades recebidas, reportando desvios e atualizando automaticamente os níveis de stock.

Responsável Financeiro

Perfil: É o utilizador encarregue da supervisão de compras e controlo orçamental.

Principais Responsabilidades:

- **Aprovação de Pedidos:** Analisar, aprovar ou rejeitar os pedidos de compra pendentes gerados pelo armazém.
- **Gestão de Fornecedores:** Registrar novos fornecedores, definir e negociar os parâmetros contratuais (preços dos produtos, custos de transporte, métodos de pagamento, prazos de entrega).
- **Avaliação de Desempenho:** Avaliar os fornecedores após a receção das encomendas (qualidade, pontualidade, comunicação, conformidade e preço).
- **Análise de Relatórios:** Consultar métricas de despesa, gráficos financeiros e estatísticas de fiabilidade dos fornecedores para otimizar os custos operacionais.

Matriz de Responsabilidades

Para resumir a interação de cada ator com as grandes funcionalidades do sistema, apresenta-se a seguinte matriz de permissões:

Funcionalidade / Operação	Administrador	Responsável de Stock	Responsável Financeiro
Criar / Suspende Utilizadores	Sim	Não	Não
Gerir Catálogo de Produtos	Sim	Sim	Não
Criar Pedidos de Compra	Sim	Sim	Não
Aprovar / Rejeitar Pedidos	Sim	Não	Sim
Gerir Fornecedores e Preços	Sim	Não	Sim
Registrar Receção de Mercadoria	Sim	Sim	Não
Avaliar Fornecedores	Sim	Não	Sim
Visualizar Relatórios Financeiros	Sim	Não	Sim

Requisitos funcionais

ID	Módulo / Categoria	Descrição do Requisito	Atores Intervenientes
RF01	Utilizadores & Segurança	Autenticação de Contas: Iniciar sessão no sistema através de um nome de utilizador e palavra-passe.	Todos os Atores
RF02	Utilizadores & Segurança	Gestão de Utilizadores: Registar novos utilizadores, definir o seu papel e redefinir palavras-passe.	Administrador
RF03	Utilizadores & Segurança	Suspensão de Acessos: Ativar ou desativar contas de utilizadores para impedir acessos imediatos ao sistema.	Administrador
RF04	Stock & Catálogo	Manutenção de Produtos: Criar, editar e suspender produtos físicos (nome, categoria, descrição, stock inicial e preço base).	Administrador, Responsável de Stock
RF05	Stock & Catálogo	Configuração de Alerta: Definir o limite de stockMínimo individual para cada produto do catálogo.	Administrador, Responsável de Stock
RF06	Stock & Catálogo	Sinalização Crítica: Detetar e realçar graficamente (alertas visuais) os produtos que se encontram em rutura de stock.	Administrador, Responsável de Stock
RF07	Fornecedores	Ficha de Fornecedores: Registar, editar e suspender fornecedores (nome, NIF único, contacto, e-mail e notas).	Administrador, Responsável Financeiro
RF08	Fornecedores	Condições Contratuais: Registar taxas logísticas (custo de transporte, valor mínimo de encomenda) e prazos acordados.	Administrador, Responsável Financeiro
RF09	Fornecedores	Catálogo de Preços Negociados: Associar produtos aos fornecedores com o respetivo preço contratual e definir o fornecedor preferencial.	Administrador, Responsável Financeiro
RF10	Fornecedores	Histórico de Avaliações: Submeter classificações de 1 a 5 estrelas em conformidade, pontualidade, comunicação, preço e qualidade.	Administrador, Responsável Financeiro
RF11	Pedidos & Aprovisionamento	Pedidos de Compra Manuais: Criar pedidos de reposição com seleção livre de produtos, quantidades e fornecedores associados.	Administrador, Responsável de Stock
RF12	Pedidos & Aprovisionamento	Sugestões Automáticas: Gerar propostas de compra automatizadas (TipoPedido.AUTOMATICO) quando um produto atinge o stock mínimo.	Administrador, Responsável de Stock
RF13	Pedidos & Aprovisionamento	Aprovação Financeira: Analisar pedidos de compra pendentes, recalcular o valor estimado e optar por aprovar ou rejeitar.	Administrador, Responsável Financeiro

RF14	Pedidos & Aprovisionamento	Emissão de Encomendas: Gerar a encomenda oficial com um código único (ex: ENQ-2026-001) imediatamente após a aprovação de um pedido.	Administrador, Responsável Financeiro
RF15	Pedidos & Aprovisionamento	Registo de Receção Física: Introduzir as quantidades de mercadoria recebidas no armazém por linha de encomenda.	Administrador, Responsável de Stock
RF16	Pedidos & Aprovisionamento	Controlo de Qualidade e Fecho: Fechar a encomenda, atualizando o stock e mudando o estado para ENTREGUE ou ENTREGUE_PARCIAL.	Administrador, Responsável de Stock
RF17	Dashboard & Relatórios	Estatísticas de Custos: Visualizar o total acumulado de despesas gerais e despesas individuais por cada fornecedor.	Administrador, Responsável Financeiro
RF18	Dashboard & Relatórios	Monitorização Operacional: Acompanhar em tempo real o volume de encomendas ativas e o número de artigos em falta.	Todos os Atores

Requisitos de qualidade

ID	Categoria	Descrição do Requisito	Implementação Técnica no VetStock
RQ01	Segurança	Controlo de Acessos Garantir que as ações críticas só são executadas por utilizadores autorizados.	Implementado através do enum Role na base de dados. As rotas do backend validam as permissões e o React esconde/mostra opções dinamicamente.
RQ02	Segurança	Autenticação e Sessão Impedir acessos não autorizados de terceiros e invalidar a sessão ao sair.	Autenticação centralizada com encriptação e lógica de logout imediato no frontend e backend que limpa o estado ativo.
RQ03	Desempenho	Tempo de Resposta Rápido Garantir que as páginas e dados carregam com latência impercetível.	Respostas da API Express optimizadas (abaixo de 300ms) e carregamento reativo ultra-rápido no frontend com Vite.
RQ04	Desempenho	Otimização de Consultas Evitar sobrecarga e múltiplas consultas repetitivas à base de dados.	Utilização de carregamento de relações (<i>eager loading</i>) nativo do Prisma ORM, trazendo apenas os dados necessários (ex: produtos + fornecedor preferencial).
RQ05	Usabilidade	Interface Responsiva Garantir que a plataforma funciona perfeitamente em computadores, tablets e telemóveis.	Uso de CSS Vanilla / Grid / Flexbox e classes CSS dinâmicas para adaptabilidade em múltiplos tamanhos de ecrã (Mobile First).
RQ06	Usabilidade	Feedback Visual Operacional Alertar o utilizador sobre estados críticos de forma simples e gráfica.	Destaque visual (vermelho/laranja) para produtos com stock abaixo do stockMinimo no painel principal (Dashboard).
RQ07	Fiabilidade	Transações Atómicas (ACID)	Execução de múltiplos passos numa

		Garantir que fluxos de dados complexos nunca ficam gravados a meio.	transação única da base de dados (prisma.\$transaction) no fecho e aprovação de encomendas.
RQ08	Fiabilidade	Consistência de Existências O inventário na plataforma deve corresponder rigorosamente à realidade física.	A receção física de encomendas incrementa automaticamente as quantidades de stock no catálogo de forma imediata e síncrona.
RQ09	Manutenibilidade	Segurança de Tipagem Prevenir erros em tempo de compilação e estruturar o desenvolvimento.	Utilização obrigatória de TypeScript em 100% da codebase (frontend e backend), assegurando a tipagem estática de todas as entidades.
RQ10	Manutenibilidade	Evolução da Base de Dados Garantir histórico e replicação uniforme da base de dados entre programadores.	Sistema de versionamento estruturado com Prisma Migrations (ficheiros SQL incrementais gerados automaticamente na pasta backend/prisma/migrations).
RQ11	Portabilidade	Reprodutibilidade do Ambiente Facilidade em colocar todo o sistema a correr em qualquer computador ou servidor.	Contentorização completa da base de dados (PostgreSQL) e painel (pgAdmin) isolada em Docker Compose, executável com um único comando.
RQ12	Portabilidade	Disponibilidade em Produção A plataforma deve estar disponível na internet de forma pública e permanente.	Deployment ativo da aplicação no domínio público www.vetstock.ONLINE , ligando o frontend e backend em produção.

Metodologia scrum

O desenvolvimento do sistema **StockVet** foi organizado de forma incremental e iterativa, dividindo-se em **7 Sprints** estratégicos. A tabela seguinte detalha as respetivas datas, o objetivo formal de cada iteração e as grandes entregas associadas:

Sprint	Período	Objetivo do Sprint (Jira Sprint Goal)
Sprint 0	11/2 - 26/2	Ter a base técnica completa do sistema pronta: backend funcional, base de dados configurada com ORM, frontend inicial, Docker e CI configurados.
Sprint 1	27/2 - 14/3	Implementar a gestão base de produtos e fornecedores com persistência em base de dados e integração frontend-backend.
Sprint 2	16/3 - 29/3	Implementar a gestão essencial de pedidos de compra, permitindo criar, consultar, editar, confirmar, rejeitar, cancelar e acompanhar o estado dos pedidos, incluindo histórico, observações e opção de rascunho.
Sprint 3	31/3 - 09/4	Consolidar o MVP do sistema através do refinamento dos módulos de Gestão de Produtos, Fornecedores e Pedidos, garantindo a correção de erros identificados em sprints anteriores e melhorando a consistência e usabilidade das interfaces existentes.

Sprint 4	20/4 - 02/5	Implementar o fluxo completo de Gestão de Encomendas, permitindo triar Pedidos de Compra para gerar Encomendas agrupadas por fornecedor, e automatizar a atualização do inventário físico no momento da receção logística.
Sprint 5	05/5 - 17/5	Melhorar a estabilidade e usabilidade da aplicação, corrigindo pedidos, sidebar, filtros, temas, pop-ups e relatórios.
Sprint 6	18/5 - 24/5	Acabar o Projeto.

Controlo de versões com git

Organizamos o repositório em três níveis hierárquicos de branches, somando um total de **mais de 65 branches** criadas ao longo do ciclo de desenvolvimento.

1. Branch Principal (main):

- Contém exclusivamente o código em estado **100% estável**, limpo e pronto para produção.
- O código desta branch está associado ao servidor de deployment contínuo, servindo diretamente a plataforma online **stockvet.ONLINE**.

2. Branches de Consolidação de Sprint (Sprint-X-Nome):

- Branches de nível intermédio criadas no início de cada ciclo (ex: Sprint-2-PedidosCompra, Sprint-4-Encomendas).

3. Branches de Funcionalidade / User Story (SCRUM-ID-US-Descricao):

- Branches de nível atómico criadas por cada programador para resolver uma tarefa específica do backlog do Jira.

Stack tecnológico

O nosso stack tecnológico é composto por:

- **Frontend (Interface): React** com **TypeScript** e **Vite**. O React permite criar uma página super dinâmica e sem carregamentos demorados. O Vite garante um arranque ultra-rápido durante o desenvolvimento e excelente performance final.
- **Backend (API): Node.js** com **Express** e **TypeScript**. Fornece uma API rápida e leve que recebe e envia dados em formato JSON.
- **Base de Dados e ORM: PostgreSQL** (com Docker Compose para ser fácil de correr em qualquer máquina) e **Prisma ORM** para fazer a ligação segura entre o código e a base de dados.

Comparação com a Alternativa: Python (Django)

Como a nossa equipa **desenvolveu o projeto do ano passado em Python e Django**, decidimos mudar para o stack atual por três motivos simples:

- **Interface fluida vs. Recarregamentos:** No projeto do ano passado com Django, o ecrã tinha de recarregar a cada clique ou alteração na base de dados. Este ano, com **React**, conseguimos que o utilizador atualize stocks, abra modais e aprove pedidos de forma imediata e sem qualquer recarregamento de página.
- **Uma única linguagem vs. Duas linguagens:** No ano passado, fomos obrigados a alternar constantemente entre Python (no backend) e JavaScript/HTML (no frontend). Neste, usamos **TypeScript em todo o lado**, o que nos poupou imenso tempo e facilitou a comunicação da equipa.

Arquitetura do sistema

Explicar como ficou a arquitetura do sistema com diagrama de arquitetura e explicar

Modelo de dados

Mostrar modelo de dados explicar

Orm e migrações

Explicar orm e migrações

Deployment e instruções de execução

Ter tudo no mesmo container do Docker e no site stockvet.ONLINE

Conclusão

Referências

Anexos